

Student 13 **Gudi Yashwanth** 192524275RD

4 / 4 submitted

Q1. String Character Frequency Counter

CODE

```
s = input().strip()
freq = {}
for ch in s:
    if ch == ' ':
        continue
    ch = ch.lower()
    freq[ch] = freq.get(ch, 0) + 1
for ch in sorted(freq.keys()):
    print(f"{ch} → {freq[ch]}")
```

INPUT

Hello world

OUTPUT

(no output)

Q2. Vowel and Consonant Classifier with Statistics

CODE

```
sentence = input()

sentence_lower = sentence.lower()

vowels = "aeiou"
vowel_count = {}
consonant_count = {}

for ch in sentence_lower:
    if ch.isalpha():
        if ch in vowels:
            vowel_count[ch] = vowel_count.get(ch, 0) + 1
        else:
            consonant_count[ch] = consonant_count.get(ch, 0) + 1

total_vowels = sum(vowel_count.values())
total_consonants = sum(consonant_count.values())

if vowel_count:
    most_frequent_vowel = max(vowel_count, key=lambda x: vowel_count[x])
    most_frequent_vowel_count = vowel_count[most_frequent_vowel]
else:
    most_frequent_vowel = None

if consonant_count:
    least_frequent_consonant = min(consonant_count, key=lambda x: consonant_count[x])
    least_frequent_consonant_count = consonant_count[least_frequent_consonant]
else:
    least_frequent_consonant = None

if total_consonants > 0:
    ratio = round(total_vowels / total_consonants, 2)
else:
    ratio = None

print(f"Vowels: {total_vowels} | Consonants: {total_consonants}", end=" ")

if most_frequent_vowel:
    print(f"Most frequent vowel: {most_frequent_vowel} ({most_frequent_vowel_count})", end=" ")
else:
    print("No vowels found", end=" ")

if least_frequent_consonant:
```

INPUT

python programming

OUTPUT

(no output)

Q3. String Compression and Run-Length Encoding

CODE

```
def compress(s):
    if not s:
        return s

    compressed = ""
    count = 1

    for i in range(1, len(s)):
        if s[i] == s[i - 1]:
            count += 1
        else:
            compressed += s[i - 1] + (str(count) if count > 1 else "")
            count = 1

    compressed += s[-1] + (str(count) if count > 1 else "")

    if len(compressed) ≥ len(s):
        return None # Not beneficial
    return compressed

def decompress(s):
    if not s:
        return s

    decompressed = ""
    i = 0

    while i < len(s):
        char = s[i]
        i += 1
        num = ""
        while i < len(s) and s[i].isdigit():
            num += s[i]
            i += 1
        decompressed += char * (int(num) if num else 1)

    return decompressed

s = input().strip()

if not s:
    print("Compressed: ")
    print("Decompressed: ")
else:
    compressed = compress(s)

    if compressed is None:
        print(f"For input '{s}': Compression not beneficial.")
        print(f"Original: {s}")
    else:
        print(f"Compressed: {compressed}")
        print(f"Decompressed: {decompress(compressed)}")
```

INPUT

aabbbccdddee

OUTPUT

(no output)

Q4. Advanced Text Statistics Analyser

CODE

```
import re
from collections import Counter

def count_syllables(word):
    word = word.lower()
    vowels = "aeiou"
    count = 0
    prev_vowel = False
    for char in word:
        if char in vowels:
            if not prev_vowel:
                count += 1
            prev_vowel = True
        else:
            prev_vowel = False
    return max(1, count)

def count_paragraphs(text):
    paragraphs = [p.strip() for p in text.strip().split('\n') if p.strip()]
    return max(1, len(paragraphs))

def get_reading_ease_label(score):
    if score ≥ 90:
        return "Very Easy"
    elif score ≥ 80:
        return "Easy"
    elif score ≥ 70:
        return "Fairly Easy"
    elif score ≥ 60:
        return "Standard"
    elif score ≥ 50:
        return "Fairly Difficult"
    elif score ≥ 30:
        return "Difficult"
    else:
        return "Very Confusing"

def analyse_text(text):
    stop_words = {"the", "is", "a", "an", "in", "of", "and", "to", "for"}

    # Count paragraphs
    num_paragraphs = count_paragraphs(text)

    # Count sentences
    sentences = re.split(r'[.!?]+', text)
    sentences = [s.strip() for s in sentences if s.strip()]
    num_sentences = len(sentences)

    # Tokenize words (remove punctuation)
    words = re.findall(r'[a-zA-Z']+', text)
    words_clean = [w.strip('').lower() for w in words if w.strip('')]
    num_words = len(words_clean)

    # Average word length
    avg_word_length = round(sum(len(w) for w in words_clean) / num_words, 1) if num_words > 0 else 0

    # Average sentence length
    avg_sentence_length = round(num_words / num_sentences, 2) if num_sentences > 0 else 0

    # Top 5 words excluding stop words
    filtered_words = [w for w in words_clean if w not in stop_words]
    word_freq = Counter(filtered_words)
    top5 = word_freq.most_common(5)
    top5_str = ", ".join(f"{w}({c})" for w, c in top5)

    # Syllable count
    total_syllables = sum(count_syllables(w) for w in words_clean)
```

```
# Flesch Reading Ease Score
if num_sentences > 0 and num_words > 0:
    flesch = 206.835 - 1.015 * (num_words / num_sentences) - 84.6 * (total_syllables / num_words)
    flesch = round(flesch, 1)
else:
    flesch = 0.0

label = get_reading_ease_label(flesch)

print(f"Words: {num_words} | Sentences: {num_sentences} | Paragraphs: {num_paragraphs}")
print(f"Avg Word Length: {avg_word_length} | Avg Sentence Length: {avg_sentence_length}")
print(f"Top 5 Words: {top5_str}")
print(f"Flesch Reading Ease: {flesch} ({label})")

text = input()
analyse_text(text)
```

INPUT

python is a powerful programming language

OUTPUT

Words: 6 | Sentences: 1 | Paragraphs: 1
Avg Word Length: 6.0 | Avg Sentence Length: 6.0
Top 5 Words: python(1), powerful(1), programming(1), language(1)
Flesch Reading Ease: 31.5 (Difficult)